

LAPORAN PRAKTIKUM STRUKTUR DATA
Single Linked List



Oleh:

Naufal Arkan Octyandi

2511533005

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU: DR. WAHYUDI, S.T, M.T

ASISTEN LABOR: SHERLY SUKMADIRA

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 04 MEI 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data yang berjudul “Single Linked List” dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum serta untuk menambah wawasan dan pemahaman mengenai konsep struktur data, khususnya penggunaan Single Linked List. Dalam praktikum ini, penulis mempelajari bagaimana cara menyimpan dan mengelola data secara dinamis menggunakan struktur berantai, serta melakukan operasi dasar seperti penambahan data (insert), penghapusan data (delete), dan penelusuran elemen (traversal) menggunakan bahasa pemrograman Java.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca dan dapat menjadi referensi tambahan dalam mempelajari struktur data, khususnya konsep Single Linked List.

Padang, 2026

Tim Penyusun

Naufal Arkan Octyandi

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 LATAR BELAKANG	1
1.2 TUJUAN PRAKTIKUM	1
1.3 MANFAAT PRAKTIKUM.....	1
BAB II PEMBAHASAN PRAKTIKUM.....	2
2.1 STRUKTUR DASAR NODE PADA SINGLE LINKED LIST.....	2
2.2 OPERASI PENAMBAHAN DATA PADA SINGLE LINKED LIST.....	2
2.3 PROSES PENCARIAN DAN PENELUSURAN DATA PADA SINGLE LINKED LIST	5
2.4 OPERASI PENGHAPUSAN DATA PADA SINGLE LINKED LIST ...	6
BAB III KESIMPULAN.....	8
3.1 KESIMPULAN.....	8
DAFTAR PUSTAKA.....	9

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Dalam perkembangan teknologi informasi, pengolahan data yang efisien menjadi hal yang penting dalam dunia pemrograman. Hal ini dapat dicapai dengan penggunaan struktur data yang tepat agar proses penyimpanan dan pengolahan data menjadi lebih optimal.

Salah satu struktur data yang digunakan adalah Single Linked List, yaitu struktur data dinamis yang terdiri dari node-node yang saling terhubung. Struktur ini memungkinkan pengelolaan data yang fleksibel, terutama dalam proses penambahan dan penghapusan elemen.

Oleh karena itu, melalui praktikum ini penulis mempelajari konsep dasar serta implementasi Single Linked List menggunakan bahasa pemrograman Java agar dapat memahami cara kerjanya dalam pengolahan data.

1.2 TUJUAN PRAKTIKUM

1. Memahami konsep dasar struktur data Single Linked List
2. Mengetahui cara kerja node dan hubungan antar elemen dalam Single Linked List
3. Mengimplementasikan Single Linked List menggunakan bahasa pemrograman Java

1.3 MANFAAT PRAKTIKUM

1. Menambah pemahaman mengenai struktur data dinamis, khususnya Single Linked List
2. Melatih kemampuan pemrograman dalam mengimplementasikan struktur data
3. Membantu memahami perbedaan antara struktur data statis dan dinamis

BAB II

PEMBAHASAN PRAKTIKUM

2.1 STRUKTUR DASAR NODE PADA SINGLE LINKED LIST

Pertama-tama kita membuat Class NodeSLL. Class ini digunakan sebagai dasar dari struktur data Single Linked List, karena setiap elemen dalam linked list direpresentasikan sebagai node dan akan digunakan sebagai komponen pada Class Class berikutnya.

Di dalam Class ini terdapat dua atribut, yaitu data dan next. Variabel data berfungsi untuk menyimpan nilai data bertipe integer, sedangkan next digunakan sebagai penunjuk ke node berikutnya. Jika node tersebut adalah node terakhir, maka next akan bernilai null.

```
1 package pekan5_2511533005;
2 import java.util.*;
3 public class NodeSLL_2511533005 {
4     //node bagian data
5     int data_3005;
6     //pointer ke node berikutnya
7     NodeSLL_2511533005 next_3005;
8     // konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511533005(int data)
10    {
11        this.data_3005 = data;
12        this.next_3005 = null;
13    }
14 }
15
16
```

2.2 OPERASI PENAMBAHAN DATA PADA SINGLE LINKED LIST

Selanjutnya dibuat Class TambahSLL yang digunakan untuk melakukan berbagai operasi penambahan data pada struktur Single Linked List. Class ini memanfaatkan Class NodeSLL sebagai dasar pembentukan node dan akan digunakan kembali pada proses pengolahan data.

Class ini memiliki beberapa method utama, yaitu insertAtFront, insertAtEnd, dan insertPos. Method insertAtFront digunakan untuk menambahkan node di bagian depan dengan menjadikan node baru sebagai head. Method insertAtEnd digunakan

untuk menambahkan node di bagian akhir dengan menelusuri hingga node terakhir, lalu menghubungkannya dengan node baru. Sedangkan insertPos digunakan untuk menambahkan node pada posisi tertentu dengan cara menyisipkan node di antara node yang ada.

Selain itu, terdapat method printList untuk menampilkan isi linked list. Dan pada bagian main program, dilakukan pembuatan linked list awal dan beberapa percobaan penambahan data di berbagai posisi untuk melihat hasil perubahannya.

```
1 package pekan5_2511533005;
2 public class TambahSLL_2511533005 {
3     public static NodeSLL_2511533005 insertAtFront(NodeSLL_2511533005 head, int value) {
4         NodeSLL_2511533005 new_node_3005 = new NodeSLL_2511533005(value);
5         new_node_3005.next_3005 = head;
6         return new_node_3005;
7     }
8     // fungsi menambahkan node di akhir SLL
9     public static NodeSLL_2511533005 insertAtEnd(NodeSLL_2511533005 head, int value) {
10        // buat sebuah node dengan sebuah nilai
11        NodeSLL_2511533005 newNode_3005 = new NodeSLL_2511533005(value);
12        // jika list kosong maka node jadi head
13        if (head == null) {
14            return newNode_3005;
15        }
16        // simpan head ke variabel sementara
17        NodeSLL_2511533005 last_3005 = head;
18        // telusuri ke node akhir
19        while (last_3005.next_3005 != null) {
20            last_3005 = last_3005.next_3005;
21        }
22        // ubah pointer
23        last_3005.next_3005 = newNode_3005;
24        return head;
25    }
26    static NodeSLL_2511533005 GetNode(int data) {
27        return new NodeSLL_2511533005(data);
28    }
29
30    static NodeSLL_2511533005 insertPos(NodeSLL_2511533005 headNode, int position, int value) {
31        NodeSLL_2511533005 head = headNode;
32        if (position < 1)
33            System.out.print("Invalid position");
34        if (position == 1) {
35            NodeSLL_2511533005 new_node_3005 = new NodeSLL_2511533005(value);
36            new_node_3005.next_3005 = head;
37            return new_node_3005;
38        } else {
39            while (position-- != 0) {
40                if (position == 1) {
```

```

38     } else {
39         while (position-- != 0) {
40             if (position == 1) {
41                 NodeSLL_2511533005 newNode_3005 = GetNode(value);
42                 newNode_3005.next_3005 = headNode.next_3005;
43                 headNode.next_3005 = newNode_3005;
44                 break;
45             }
46             headNode = headNode.next_3005;
47         }
48         if (position != 1)
49             System.out.print("Posisi di luar jangkauan");
50     }
51     return head;
52 }
53 public static void printList(NodeSLL_2511533005 head) {
54     NodeSLL_2511533005 curr_3005 = head;
55     while (curr_3005.next_3005 != null) {
56         System.out.print(curr_3005.data_3005+"-->");
57         curr_3005 = curr_3005.next_3005;
58     }
59     if (curr_3005.next_3005==null) {
60         System.out.print(curr_3005.data_3005);
61     }
62     System.out.println();
63 }
64
65 public static void main(String[] args) {
66     // buat linked list 2->3->5->6
67     NodeSLL_2511533005 head_3005 = new NodeSLL_2511533005(2);
68     head_3005.next_3005 = new NodeSLL_2511533005(3);
69     head_3005.next_3005.next_3005 = new NodeSLL_2511533005(5);
70     head_3005.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(6);
71     // cetak list asli
72     System.out.print("Senarai berantai awal:");
73     printList(head_3005);
74     // tambahkan node baru di depan
75     System.out.print("tambah 1 simpul di depan: ");
76     int data = 1;
77     head_3005 = insertAtFront(head_3005, data);

```

```

54     NodeSLL_2511533005 curr_3005 = head;
55     while (curr_3005.next_3005 != null) {
56         System.out.print(curr_3005.data_3005+"-->");
57         curr_3005 = curr_3005.next_3005;
58     }
59     if (curr_3005.next_3005==null) {
60         System.out.print(curr_3005.data_3005);
61     }
62     System.out.println();
63 }
64
65 public static void main(String[] args) {
66     // buat linked list 2->3->5->6
67     NodeSLL_2511533005 head_3005 = new NodeSLL_2511533005(2);
68     head_3005.next_3005 = new NodeSLL_2511533005(3);
69     head_3005.next_3005.next_3005 = new NodeSLL_2511533005(5);
70     head_3005.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(6);
71     // cetak list asli
72     System.out.print("Senarai berantai awal:");
73     printList(head_3005);
74     // tambahkan node baru di depan
75     System.out.print("tambah 1 simpul di depan: ");
76     int data = 1;
77     head_3005 = insertAtFront(head_3005, data);
78     // cetak update list
79     printList(head_3005);
80     // tambahkan node baru di belakang
81     System.out.print("tambah 1 simpul di belakang: ");
82     int data2 = 7;
83     head_3005 = insertAtEnd(head_3005, data2);
84     // cetak update list
85     printList(head_3005);
86     System.out.print("tambah 1 simpul ke data 4: ");
87     int data3 = 4;
88     int pos=4;
89     head_3005 = insertPos(head_3005,pos,data3);
90     // cetak update list
91     printList(head_3005);
92 }
93 }

```

2.3 PROSES PENCARIAN DAN PENELUSURAN DATA PADA SINGLE LINKED LIST

Selanjutnya dibuat Class pencarianSLL yang berfungsi untuk melakukan pencarian data dan penelusuran pada Single Linked List. Class ini juga memanfaatkan NodeSLL sebagai struktur dasar node.

Pada class ini terdapat method searchKey yang digunakan untuk mencari data tertentu dalam linked list. Method ini bekerja dengan menelusuri setiap node dari head hingga akhir. Jika data yang dicari ditemukan, maka method akan mengembalikan nilai *true*, dan jika tidak ditemukan akan mengembalikan *false*. Selain itu, terdapat method traversal yang digunakan untuk menampilkan seluruh isi linked list dengan cara menelusuri node satu per satu dari awal hingga akhir.

Pada bagian main, dibuat contoh linked list dengan beberapa data, kemudian dilakukan proses penelusuran untuk menampilkan isi list, serta pencarian data tertentu untuk mengetahui apakah data tersebut ada di dalam linked list atau tidak.

```
1 package pekan5_2511533005;
2
3 public class pencarianSLL_2511533005 {
4     static boolean searchKey (NodeSLL_2511533005 head, int Key) {
5         NodeSLL_2511533005 curr_3005= head;
6         while (curr_3005 != null) {
7             if(curr_3005.data_3005==Key)
8                 return true;
9             curr_3005 = curr_3005.next_3005; }
10        return false; }
11    public static void traversal (NodeSLL_2511533005 head) {
12        //mulai dari head
13        NodeSLL_2511533005 curr_3005 = head;
14        //telusuri sampai pointer null
15        while (curr_3005 != null) {
16            System.out.print(" " + curr_3005.data_3005);
17            curr_3005 = curr_3005.next_3005;}
18        System.out.println(); }
19
20    public static void main (String[] args) {
21        NodeSLL_2511533005 head_3005 = new NodeSLL_2511533005(14);
22        head_3005.next_3005 = new NodeSLL_2511533005(21);
23        head_3005.next_3005.next_3005 = new NodeSLL_2511533005(13);
24        head_3005.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(30);
25        head_3005.next_3005.next_3005.next_3005.next_3005= new NodeSLL_2511533005(10);
26        System.out.print("Penelesuran SLL : ");
27        traversal (head_3005);
28        //data yang akan dicari
29        int Key=30;
30        System.out.print("cari data " +Key+ " = ");
31        if (searchKey(head_3005, Key))
32            System.out.println("Ketemu");
33        else
34            System.out.println("Tidak ada");
35    }
36
37 }
38
39
```


2.4 OPERASI PENGHAPUSAN DATA PADA SINGLE LINKED LIST

Selanjutnya dibuat class HapusSLL yang berfungsi untuk melakukan operasi penghapusan data pada Single Linked List dengan menggunakan NodeSLL sebagai dasar node.

Pada class ini terdapat beberapa method utama, yaitu deleteHead, removeLastNode, dan deleteNode. Method deleteHead digunakan untuk menghapus node pertama dengan cara memindahkan head ke node berikutnya. Method removeLastNode digunakan untuk menghapus node terakhir dengan menelusuri hingga node sebelum terakhir, lalu menghapus hubungan ke node terakhir tersebut. Sedangkan deleteNode digunakan untuk menghapus node pada posisi tertentu dengan cara menelusuri hingga posisi yang dimaksud, kemudian mengubah pointer agar node tersebut terlepas dari linked list. Selain itu, terdapat method printList untuk menampilkan isi linked list setelah dilakukan penghapusan.

Pada bagian main, dibuat contoh linked list, kemudian dilakukan beberapa operasi penghapusan seperti menghapus node di depan, di belakang, dan pada posisi tertentu untuk melihat perubahan yang terjadi pada linked list.

```
1 package pekan5_2511533005;
2 public class HapusSLL_2511533005 {
3     // fungsi untuk menghapus head
4     public static NodeSLL_2511533005 deleteHead(NodeSLL_2511533005 head) {
5         // jika SLL kosong
6         if (head == null)
7             return null;
8         // pindahkan head ke node berikutnya
9         head = head.next_3005;
10        // Return head baru
11        return head;
12    }
13    // fungsi menghapus node terakhir SLL
14    public static NodeSLL_2511533005 removeLastNode(NodeSLL_2511533005 head) {
15        // jika list kosong, return null
16        if (head == null) {
17            return null;
18        }
19        // jika list satu node, hapus node dan return null
20        if (head.next_3005 == null) {
21            return null;
22        }
23        // temukan node terakhir ke dua
24        NodeSLL_2511533005 secondLast_3005 = head;
25        while (secondLast_3005.next_3005.next_3005 != null) {
26            secondLast_3005 = secondLast_3005.next_3005;
27        }
28        // hapus node terakhir
29        secondLast_3005.next_3005 = null;
30        return head;
31    }
32    // fungsi menghapus node di posisi tertentu
33    public static NodeSLL_2511533005 deleteNode(NodeSLL_2511533005 head, int position) {
34        NodeSLL_2511533005 temp_3005 = head;
35        NodeSLL_2511533005 prev_3005 = null;
36        // jika linked list null
37        if (temp_3005 == null)
38            return head;
39        // kasus 1: head dihapus
40        if (position == 1) {
```

```

33 public static NodeSLL_2511533005 deleteNode(NodeSLL_2511533005 head, int position) {
34     NodeSLL_2511533005 temp_3005 = head;
35     NodeSLL_2511533005 prev_3005 = null;
36     // jika linked list null
37     if (temp_3005 == null)
38         return head;
39     // kasus 1: head dihapus
40     if (position == 1) {
41         head = temp_3005.next_3005;
42         return head;
43     }
44     // kasus 2: menghapus node di tengah
45     // telusuri ke node yang dihapus
46     for (int i = 1; temp_3005 != null && i < position; i++) {
47         prev_3005 = temp_3005;
48         temp_3005 = temp_3005.next_3005;
49     }
50     // jika ditemukan, hapus node
51     if (temp_3005 != null) {
52         prev_3005.next_3005 = temp_3005.next_3005;
53     } else {
54         System.out.println("Data tidak ada");
55     }
56     return head;
57 }
58 // fungsi mencetak SLL
59 public static void printList(NodeSLL_2511533005 head) {
60     NodeSLL_2511533005 curr_3005 = head;
61     while (curr_3005.next_3005 != null) {
62         System.out.print(curr_3005.data_3005+"->");
63         curr_3005 = curr_3005.next_3005;
64     }
65     if (curr_3005.next_3005 == null) {
66         System.out.print(curr_3005.data_3005);
67     }
68     System.out.println();
69 }
70 // kelas main
71 public static void main(String[] args) {
72     // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
73     NodeSLL_2511533005 head = new NodeSLL_2511533005(1);
74     head.next_3005 = new NodeSLL_2511533005(2);
75     head.next_3005.next_3005 = new NodeSLL_2511533005(3);
76     head.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(4);
77     head.next_3005.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(5);
78     head.next_3005.next_3005.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(6);
79     // cetak list awal
80 }

```

```

64 public static void main(String[] args) {
65     // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
66     NodeSLL_2511533005 head = new NodeSLL_2511533005(1);
67     head.next_3005 = new NodeSLL_2511533005(2);
68     head.next_3005.next_3005 = new NodeSLL_2511533005(3);
69     head.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(4);
70     head.next_3005.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(5);
71     head.next_3005.next_3005.next_3005.next_3005.next_3005 = new NodeSLL_2511533005(6);
72     // cetak list awal
73     System.out.println("list awal: ");
74     printList(head);
75     // hapus head
76     head = deleteHead(head);
77     System.out.println("List setelah head dihapus: ");
78     printList(head);
79     // hapus node terakhir
80     head = removeLastNode(head);
81     System.out.println("List setelah simpul terakhir di hapus: ");
82     printList(head);
83     // Deleting node at position 2
84     int position = 2;
85     head = deleteNode(head, position);
86     // Print list after deletion
87     System.out.println("List setelah posisi 2 dihapus: ");
88     printList(head);
89 }
90 }
91 }

```

BAB III

KESIMPULAN

3.1 KESIMPULAN

Pada praktikum ini, penulis mempelajari implementasi struktur data Single Linked List menggunakan bahasa pemrograman Java. Struktur ini terdiri dari node-node yang saling terhubung, di mana setiap node menyimpan data dan pointer ke node berikutnya. Praktikum dimulai dengan pembuatan node sebagai dasar pembentukan linked list, kemudian dilanjutkan dengan berbagai operasi seperti penambahan data di depan, belakang, dan posisi tertentu. Selain itu, dilakukan juga proses penelusuran untuk menampilkan data serta pencarian untuk mengetahui keberadaan suatu nilai dalam linked list. Selanjutnya, dilakukan operasi penghapusan data, baik pada bagian depan, belakang, maupun posisi tertentu, dengan cara mengatur kembali hubungan antar node. Secara keseluruhan, praktikum ini membantu memahami cara kerja Single Linked List dalam mengelola data secara dinamis melalui berbagai operasi dasar.

DAFTAR PUSTAKA

Programiz. *Linked List Data Structure*.

<https://www.programiz.com/dsa/linked-list>

LearnDSA. *Linked Lists Tutorial*.

<https://learndsa.org/topics/linked-lists>

InfoWorld. *Singly Linked List in Java*.

<https://www.infoworld.com/article/2266416/data-structures-and-algorithms-in-java-part-4-singly-linked-lists.html>